Programming Assignment: Reading, Grouping, and Analyzing JSON/CSV Data

Overview

In this assignment, you will write a program that reads street and traffic information from either a **JSON file** or a **CSV file**.

The user will decide which file format the program should read.

Regardless of which file is selected, the data must be converted into objects and stored in an `ArrayList` or Array

After the data has been loaded, your program will group and analyze the street information.

The JSON and CSV files contain the same type of information, but they are stored using different file formats.

Data Fields

Each record contains the following information:

Field	Description
`street_id`	Unique identifier for the street record
`city`	Name of the city
`city_population`	Population of the city
`street_name`	Name of the street
`cars_traveled_daily`	Estimated number of cars traveling the street each day
`speed_limit_mph`	Speed limit for the street

The provided data files contain approximately **50,000 street records**.

Program Requirements

1. Create a Street Class

Create a class that represents one street record.

The class should contain fields corresponding to the data in the files.

For example:

Street
---
streetId
city
cityPopulation
streetName
carsTraveledDaily
speedLimitMph

Include an appropriate constructor, getters, and any other methods you believe are useful.

---

2. Allow the User to Select the File Type

When the program begins, display a menu similar to:

Street Traffic Data Analyzer

1. Read CSV File
2. Read JSON File

Select file type:

The user must be able to select either the CSV or JSON version of the dataset.

For example:

Select file type: 2

Reading JSON file...
50,000 street records loaded.

The two choices should ultimately produce the **same analysis results**.

Part 1 – Reading the Data

CSV

If the user selects CSV:

1. Open the CSV file.
2. Skip the header row.
3. Read each record.
4. Separate the record into its individual fields.
5. Convert numeric values to the appropriate numeric data types.
6. Create a `Street` object.
7. Store the object in an `ArrayList<Street>`.

Your program should **not** simply keep the CSV records as Strings.

JSON

If the user selects JSON:

1. Open the JSON file.
2. Parse the JSON data.
3. Convert each JSON record into a `Street` object.
4. Store the objects in an `ArrayList<Street>`.

You may use the JSON library demonstrated in class.

Part 2 – Grouping the Data

Once the file has been read, all remaining operations must use the objects stored in the `ArrayList`.

Do not reread the original file to calculate your results.

Group the street records by **city**.

For example:

Ashfield
- Main Street
- Oak Street
- River Road
- 1st Avenue

Clearview
- Main Street
- Pine Street
- Broadway

Silverlake
- Main Street
- Cedar Street
- Park Avenue

Notice that street names such as `Main Street` can occur in multiple cities. Therefore, **street name alone cannot be used to determine the city**.

Part 3 – Analyze the Data

Using the objects you loaded and grouped, calculate the following information.

A. Number of Cities

Determine how many unique cities exist in the dataset.

Example:

Number of Cities: 237

Do not count the same city multiple times.

B. Number of Street Records

Determine the total number of street records that were loaded.

Example:

Number of Streets: 50,000

C. Average Speed Limit

Calculate the average speed limit across all street records.

Example:

Average Speed Limit: 43.72 mph

Display the answer to **two decimal places**.

D. Average Number of Streets per City

Calculate the average number of street records associated with each city.

Conceptually:

Average Streets per City =
Total Number of Street Records

Number of Unique Cities

Example:

Average Streets per City: 210.97

Display the answer to **two decimal places**.

Part 4 – City Group Summary

Demonstrate that your data was successfully grouped by displaying a summary for each city.

For each city, display:

- City name
- City population
- Number of street records
- Average speed limit

For example:

```
CITY SUMMARY
------------------------------------------------------
City            Population     Streets     Avg Speed
------------------------------------------------------
Ashfield          82,500         187         41.23
Clearview         156,000          293          44.65
Silverlake        42,700         121         38.93
...
```

The population should not be added together simply because it appears on multiple street records. It represents the population of the city.

Final Program Summary

At the end of the program, display an overall summary similar to:

```
================================================
        STREET TRAFFIC DATA SUMMARY
================================================
```

File Type: JSON
Records Loaded: 50,000

Number of Cities: 237
Number of Streets: 50,000
Average Speed Limit: 43.72 mph
Average Streets per City: 210.97

==========

If the user runs the program again and selects the CSV file, the calculated results should be the same.

Important Requirements

Your program must:

1. Allow the user to choose between the JSON and CSV files.
2. Read the selected file.
3. Convert every record into a `Street` object.
4. Store the objects in an `ArrayList<Street>`.
5. Group the objects by city.
6. Determine the number of unique cities.
7. Determine the total number of street records.
8. Calculate the average speed limit.
9. Calculate the average number of streets per city.
10. Produce a city-by-city summary.
11. Produce an overall summary.
12. Format averages to two decimal places.

After the data has been loaded, calculations must be performed using the objects in memory rather than rereading the source file.

Suggested Program Structure

Your program might contain methods similar to:

readCSV(...)
readJSON(...)
groupByCity(...)
calculateAverageSpeed(...)
displayCitySummary(...)
displayOverallSummary(...)

You do not have to use these exact method names, but your program should be divided into logical methods.

Avoid placing the entire solution inside `main()`.

Grading

Requirement	Points
`Street` class correctly designed	10
User can select CSV or JSON	5
CSV file correctly read	10
JSON file correctly read	10
Data converted into `Street` objects	10
Objects stored in an `ArrayList`	10
Data correctly grouped by city	15
Number of cities calculated correctly	5
Number of streets calculated correctly	5
Average speed limit calculated correctly	5
Average streets per city calculated correctly	5
City summary and overall output	5
Code organization and readability	5

Total	**100**

Files Provided

You will receive:

city_street_traffic_50000.csv
city_street_traffic_50000.json

Both files contain the same dataset in different formats.

Your program must work correctly with either file.